

MACHINE TRANSLATION AND IMPLEMENTATION OF TRANSFORMERS

*Submitted in partial fulfilment of
the credit requirements for the degree of
Bachelor of Technology with Honours
in Computer Science and Engineering
(Artificial Intelligence and Machine Learning)*

Padavinangady Nakul Bhat

Registration N^o 230962212
School of Computer Engineering
Manipal Institute of Technology

July–December 2025

Abstract

This report covers the topics presented in the “Introductions to Large Language Models” course taught by Tanmoy Chakraborty and Soumen Chakrabarti, available on NPTEL. As a graded assignment, the report follows a set of seven questions, and accompanies a code base available in the repository linked below. The report is split into seven sections, each corresponding to a question from the assignment. Questions will be quoted in full and italicized at the start of each section, followed by explanations and code listings where appropriate.

Note To Readers

A repository accompanying this report is available at the author’s GitHub homepage. It is accessible to the public and can be navigated to by clicking or scanning the QR Code on the right.

The repository contains source code used for the development of models described in this report in the `code/` directory. The repository also contains the $\text{\LaTeX}$ source code used for the compilation of this report in the `report/` directory. A `flake.nix` and `flake.lock` have also been provided for readers familiar with Nix to reproduce the development environment.



QR Code linking to
accompanying
repository

Work Attribution

All work for this project has been carried out without any AI assistance or code/text generation. A work history is provided at the end of the report as a proof of work. This work was solely developed by me, Padavinangady Nakul Bhat, without any undeclared assistance. All sources used for references have been declared in the report, and are also available as a bibliography file in `report/` directory.

This work is licensed under a Creative Commons “Attribution 4.0 International” license.

doclicense-CC-by-88x31.pdf

```

1 class Encoder(nn.Module):
2     def __init__(self, input_size, hidden_size, dropout_p = 0.1):
3         super(Encoder, self).__init__()
4         self.hidden_size = hidden_size
5         self.embedding = nn.Embedding(input_size, hidden_size)
6         self.gru = nn.GRU(hidden_size, hidden_size, batch_first=True)
7         self.dropout = nn.Dropout(dropout_p)
8
9     def forward(self, input):
10        embedded = self.dropout(self.embedding(input))
11        output, hidden = self.gru(embedded)
12        return output, hidden

```

Listing 1: Encoder of the Seq2Seq Model

1 Sequence-to-Sequence Machine Translation Model

Implement a Transformer-based sequence-to-sequence model in PyTorch for machine translation (English $\rightarrow$ French). Explain the encoder-decoder structure, attention mechanism, and training process. Provide code snippets and discuss evaluation metrics such as BLEU score.

The full working implementation is available in `code/seq2seq.py`. Only important parts of the code are discussed below to provide an illustration of the working. The code largely follows the flow outlined in the PyTorch Documentation [4]. The dataset was downloaded from Tatoeba [5], and preprocessed before usage. Preprocessing included extracting pairs of English sentences with French translations, and converting Unicode strings into ASCII strings.

First, we define a class to help build a vocabulary and get one-hot encoded vectors. The `class Lang` is responsible for the ingestion of sentences and the storage of dictionaries holding the indices of words present in the language. A simple tokenizer is used, and tokens are split by whitespaces.

The encoder-decoder architecture is one of three possible transformer architectures [1] and is especially suited for summarization and machine translation as there is no restriction on both the input and output length. It uses the encoder part to calculate a dense context vector that effectively captures all the data from the input vector. The encoder we have used is an RNN with a Gated Recurrent Unit that allows it to have better context and memory. The encoder code is given in Listing 1.

The decoder part of the transformer consists of another RNN-GRU, and uses attention along with already generated tokens to get a probability distribution of possible continuations. The decoder can work in two ways, by using previous tokens as mentioned earlier, or by “teacher-forcing” where we can force the next step of attention to use tokens provided by a ground truth. Teacher-forcing allows a model to train faster

and make less mistakes by not allowing mistakes to propagate far into the model. Code for the decoder can be found in Listing 2.

```
1 class AttnDecoderRNN(nn.Module):
2     def __init__(self, hidden_size, output_size, dropout_p=0.1):
3         super(AttnDecoderRNN, self).__init__()
4         self.embedding = nn.Embedding(output_size, hidden_size)
5         self.attention = BahdanauAttention(hidden_size)
6         self.gru = nn.GRU(2 * hidden_size, hidden_size,
7             ↪ batch_first=True)
8         self.out = nn.Linear(hidden_size, output_size)
9         self.dropout = nn.Dropout(dropout_p)
10
11     def forward(self, encoder_outputs, encoder_hidden,
12         ↪ target_tensor=None):
13         batch_size = encoder_outputs.size(0)
14         decoder_input = torch.empty(batch_size, 1, dtype=torch.long,
15             ↪ device=DEVICE).fill_(SOS_TOKEN)
16         decoder_hidden = encoder_hidden
17         decoder_outputs = []
18         attentions = []
19
20         for i in range(MAX_STEPS):
21             decoder_output, decoder_hidden, attn_weights =
22                 ↪ self.forward_step(
23                 decoder_input, decoder_hidden, encoder_outputs
24             )
25             decoder_outputs.append(decoder_output)
26             attentions.append(attn_weights)
27
28             if target_tensor is not None:
29                 # Teacher forcing: Feed the target as the next input
30                 decoder_input = target_tensor[:, i].unsqueeze(1) #
31                 ↪ Teacher forcing
32             else:
33                 # Without teacher forcing: use its own predictions as
34                 ↪ the next input
35                 _, topi = decoder_output.topk(1)
36                 decoder_input = topi.squeeze(-1).detach() # detach from
37                 ↪ history as input
38
39         decoder_outputs = torch.cat(decoder_outputs, dim=1)
40         decoder_outputs =
41             ↪ torch.nn.functional.log_softmax(decoder_outputs, dim=-1)
```

```

34     attentions = torch.cat(attentions, dim=1)
35
36     return decoder_outputs, decoder_hidden, attentions
37
38
39 def forward_step(self, input, hidden, encoder_outputs):
40     embedded = self.dropout(self.embedding(input))
41
42     query = hidden.permute(1, 0, 2)
43     context, attn_weights = self.attention(query, encoder_outputs)
44     input_gru = torch.cat((embedded, context), dim=2)
45
46     output, hidden = self.gru(input_gru, hidden)
47     output = self.out(output)
48
49     return output, hidden, attn_weights

```

Listing 2: Attention Decoder of the Seq2Seq Model

Attention is a mechanism to control the tokens that a model considers relevant and important. Attention allows a decoder access to all the states of the encoder, and selectively picks the most important ones.

For every generated token, attention calculates an alignment score, called the *attention weight*. Instead of compressing all information from the encoder into a single fixed-length vector, attention computes a weighted combination of all encoder hidden states. This allows the decoder to dynamically focus on different parts of the input sequence at each decoding step.

In this work, we specifically use *Bahdanau attention* [2], which is an additive attention mechanism. Given the encoder hidden states h_i and the current decoder hidden state s_t , an alignment score is computed as

$$e_{t,i} = v_a^\top \tanh(W_a s_t + U_a h_i),$$

where W_a , U_a , and v_a are learnable parameters. These scores are normalized using a softmax function to obtain attention weights

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}.$$

The context vector c_t is then computed as a weighted sum of encoder hidden states

$$c_t = \sum_i \alpha_{t,i} h_i.$$

This context vector is concatenated with the decoder hidden state and passed to the output layer to generate the next token. The attention implementation used in this project is shown in Listing 3.

```

1 class BahdanauAttention(nn.Module):
2     def __init__(self, hidden_size):
3         super(BahdanauAttention, self).__init__()
4         self.Wa = nn.Linear(hidden_size, hidden_size)
5         self.Ua = nn.Linear(hidden_size, hidden_size)
6         self.Va = nn.Linear(hidden_size, 1)
7
8     def forward(self, query, keys):
9         scores = self.Va(torch.tanh(self.Wa(query) + self.Ua(keys)))
10        scores = scores.squeeze(2).unsqueeze(1)
11
12        weights = torch.nn.functional.softmax(scores, dim=-1)
13        context = torch.bmm(weights, keys)
14
15    return context, weights

```

Listing 3: Bahdanau Attention used in the Seq2Seq Model

The training process consists of minimizing the negative log-likelihood loss between the predicted output sequence and the ground truth translation. Given a target sentence $y = (y_1, \dots, y_T)$, the loss is defined as

$$\mathcal{L} = - \sum_{t=1}^T \log p(y_t \mid y_{<t}, x),$$

where x is the input English sentence. The model is trained using the Adam optimizer, and gradients are computed using backpropagation through time. Teacher-forcing is applied with a fixed probability, allowing the decoder to alternate between using its own predictions and ground-truth tokens during training. This improves convergence speed and stability.

During evaluation, the model is switched to inference mode and generates translations autoregressively without teacher-forcing. To quantitatively assess translation quality, we use the BLEU (Bilingual Evaluation Understudy) score, which measures the overlap of n -grams between the generated translation and a reference translation. BLEU is defined as

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right),$$

where p_n denotes the precision of n -grams, w_n are uniform weights, and BP is a brevity penalty that discourages overly short translations. BLEU provides a standardized and widely-used metric for comparing machine translation systems, and is well-suited for evaluating the English-to-French task considered here.

2 Transformers and Multi-Head Attention

Explain the architecture of a Transformer model and implement a minimal Transformer encoder using PyTorch. Highlight the role of multi-head attention and position-wise feed-forward networks. Provide code snippets to support your answer.

The Transformer architecture was introduced to address the limitations of recurrent models in sequence modeling tasks, most notably their sequential computation and difficulty in capturing long-range dependencies. Unlike RNN-based models, Transformers rely entirely on attention mechanisms and dispense with recurrence and convolution altogether [6]. This allows for full parallelization during training and more efficient modeling of global context.

A Transformer is composed of stacked encoder and decoder blocks. In this section, we focus on a minimal Transformer encoder, which is sufficient to illustrate the core architectural components. Each encoder block consists of two main sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network. Both sub-layers are wrapped with residual connections followed by layer normalization. Details of the self-attention and multi-head attention mechanisms are discussed in more depth in Section 4.

Following the attention sub-layer, each encoder block contains a position-wise feed-forward network. This network is applied independently and identically to each position in the sequence, and consists of two linear transformations with a non-linear activation function in between

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

The feed-forward network increases the representational capacity of the model and introduces non-linearity after the attention operation.

Since the Transformer contains no recurrence or convolution, positional information must be injected explicitly. This is achieved using positional encodings, which are added to the input embeddings to provide the model with information about token order. A minimal Transformer encoder implemented in PyTorch is shown in Listing 4.

3 Sinusoidal Positional Encoding

Discuss the importance of positional encoding in Transformers. Implement sinusoidal positional encoding in PyTorch and demonstrate how it integrates with input embeddings. Explain why positional information is critical for sequence modeling.

Transformers process input sequences in parallel and do not contain any inherent notion of token order. As a result, positional information must be explicitly provided to the model in order to preserve the structure of sequential data. Without positional encoding, a Transformer would treat the input as a bag of tokens, making it incapable of distinguishing between different permutations of the same sequence. Positional encoding therefore plays a critical role in sequence modeling tasks such as machine translation, language modeling, and text summarization.

```

1 class TransformerEncoderLayer(nn.Module):
2     def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
3         super().__init__()
4         self.self_attn = nn.MultiheadAttention(d_model, num_heads,
5         ↪ dropout=dropout)
6         self.ffn = nn.Sequential(
7             nn.Linear(d_model, d_ff),
8             nn.ReLU(),
9             nn.Linear(d_ff, d_model)
10        )
11        self.norm1 = nn.LayerNorm(d_model)
12        self.norm2 = nn.LayerNorm(d_model)
13        self.dropout = nn.Dropout(dropout)
14
15    def forward(self, x):
16        # x: (seq_len, batch_size, d_model)
17        attn_output, _ = self.self_attn(x, x, x)
18        x = self.norm1(x + self.dropout(attn_output))
19        ffn_output = self.ffn(x)
20        x = self.norm2(x + self.dropout(ffn_output))
21        return x

```

Listing 4: Minimal Transformer Implmentation

The original Transformer architecture introduces *sinusoidal positional encodings* [6], which inject position-dependent signals into the input embeddings using deterministic functions. These encodings are added directly to the word embeddings, allowing the model to leverage both semantic and positional information simultaneously. Since the positional encodings have the same dimensionality as the embeddings, this integration does not increase the model complexity.

For a given position pos and embedding dimension i , the sinusoidal positional encoding is defined as

$$\begin{aligned} \text{PE}(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \\ \text{PE}(pos, 2i + 1) &= \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \end{aligned}$$

where d_{model} denotes the embedding dimension. Sine functions are used for even indices and cosine functions for odd indices, resulting in a unique positional encoding for each position. This formulation ensures that positions that are close in the sequence have similar encodings, while still allowing the model to distinguish distant positions.

An important property of sinusoidal positional encodings is that they allow the model to generalize to sequence lengths longer than those seen during training. Since the encodings are generated using fixed mathematical functions rather than learned parameters, they can be computed for arbitrary sequence lengths without retraining. Additionally, relative positions can be expressed as linear functions of these encodings, which is beneficial for attention-based models.

In practice, positional encodings are added element-wise to the input embeddings before being passed to the Transformer encoder. If $E \in \mathbb{R}^{T \times d_{\text{model}}}$ denotes the token embeddings and $P \in \mathbb{R}^{T \times d_{\text{model}}}$ the positional encodings, the combined input is given by

$$X = E + P.$$

This simple operation allows every attention layer in the network to access positional information implicitly.

A PyTorch implementation of sinusoidal positional encoding is shown in Listing 5. The code constructs a positional encoding matrix once and registers it as a buffer, ensuring that it is not treated as a learnable parameter. This positional encoding is then added to the input embeddings at each forward pass, providing positional information into the Transformer pipeline.

4 Multi-Head Self-Attention

Describe the working of multi-head self-attention. Write PyTorch code to implement scaled dot-product attention and extend it to multi-head attention. Explain how multiple heads improve representation learning.

Multi-head self-attention is the central component of the Transformer architecture and is responsible for modeling dependencies between tokens within a sequence. Unlike recurrent models, self-attention allows every token to directly attend to all other tokens in

```

1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model, max_len=5000):
3         super().__init__()
4         pe = torch.zeros(max_len, d_model)
5         position = torch.arange(0, max_len).unsqueeze(1)
6         div_term = torch.exp(
7             torch.arange(0, d_model, 2) * (-math.log(10000.0) /
8                 ↪ d_model))
9         pe[:, 0::2] = torch.sin(position * div_term)
10        pe[:, 1::2] = torch.cos(position * div_term)
11        pe = pe.unsqueeze(1) # (max_len, 1, d_model)
12        self.register_buffer("pe", pe)
13
14    def forward(self, x):
15        # x: (seq_len, batch_size, d_model)
16        x = x + self.pe[:x.size(0)]
17        return x

```

Listing 5: Sinusoidal Positional Encoding

the sequence, enabling efficient capture of both short-range and long-range relationships. This mechanism forms the basis of the Transformer model introduced in [6].

Self-attention operates by projecting an input sequence into three distinct representations: queries, keys, and values. Given an input matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$, these projections are computed using learned weight matrices

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where W_Q , W_K , and W_V are trainable parameters. The attention mechanism then computes pairwise similarity scores between queries and keys to determine how much each token should attend to every other token.

The core operation is known as *scaled dot-product attention*, which is defined as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V,$$

where d_k is the dimensionality of the key vectors. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large, which stabilizes training by keeping gradients well-behaved. The softmax operation ensures that the attention weights form a valid probability distribution.

While a single attention operation is powerful, using multiple attention heads significantly improves the expressiveness of the model. Multi-head attention splits the input representations into h smaller subspaces and performs attention independently in each

subspace. Each head attends to the sequence using different learned projections, allowing the model to capture diverse relationships such as syntactic structure, semantic similarity, or positional alignment.

Formally, for h attention heads, the multi-head attention operation is defined as

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O,$$

where each head is computed as

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i),$$

and W_O is a learned output projection matrix. By concatenating the outputs of multiple heads, the model combines information from different representation subspaces into a single unified embedding.

A PyTorch implementation of scaled dot-product attention and its extension to multi-head self-attention is shown in Listing 6. The code illustrates how queries, keys, and values are reshaped to support multiple heads, how attention is computed in parallel, and how the outputs are concatenated and projected back to the original embedding dimension.

5 Sentiment Analysis

Design and implement a PyTorch pipeline to train a Transformer model for a text classification task (e.g., sentiment analysis). Include dataset preprocessing, model definition, training loop, and evaluation. Discuss challenges such as overfitting and computational cost.

Sentiment analysis is a supervised text classification task in which the goal is to assign a discrete sentiment label, such as positive or negative, to a given input sentence. Transformer-based models are particularly well-suited for this task, as self-attention enables the model to capture contextual dependencies across an entire sequence. In this section, we design and implement a PyTorch pipeline for training a Transformer encoder on a sentiment classification dataset, following the implementation outlined in [3].

The pipeline begins with dataset preprocessing. Raw text sentences are first tokenized using a simple whitespace-based tokenizer, and a vocabulary is constructed from the training corpus. Each token is mapped to a unique index, allowing sentences to be converted into sequences of integers. Since Transformer models operate on fixed-size batches, all sequences are padded to a common length. A corresponding padding mask is created to ensure that padded tokens do not contribute to the attention scores or pooled representations during training.

The model architecture consists of an embedding layer followed by positional encoding and a Transformer encoder. The embedding layer maps token indices to dense vectors, while positional encoding injects word-order information that is otherwise absent from the attention mechanism. These embeddings are then passed through multiple layers of a Transformer encoder with multi-head self-attention, allowing the model to learn contextualized representations of each token in the input sequence.

```

1 class MultiHeadSelfAttention(nn.Module):
2     def __init__(self, d_model, num_heads):
3         super().__init__()
4         assert d_model % num_heads == 0
5
6         self.num_heads = num_heads
7         self.d_k = d_model // num_heads
8
9         self.W_q = nn.Linear(d_model, d_model)
10        self.W_k = nn.Linear(d_model, d_model)
11        self.W_v = nn.Linear(d_model, d_model)
12        self.W_o = nn.Linear(d_model, d_model)
13
14        def scaled_dot_product_attention(self, Q, K, V):
15            # Q, K, V: (batch, heads, seq_len, d_k)
16            scores = torch.matmul(Q, K.transpose(-2, -1)) / (self.d_k **
17                ↪ 0.5)
18            weights = torch.softmax(scores, dim=-1)
19            return torch.matmul(weights, V)
20
21        def forward(self, x):
22            # x: (batch, seq_len, d_model)
23            B, T, _ = x.size()
24
25            Q = self.W_q(x).view(B, T, self.num_heads,
26                ↪ self.d_k).transpose(1, 2)
27            K = self.W_k(x).view(B, T, self.num_heads,
28                ↪ self.d_k).transpose(1, 2)
29            V = self.W_v(x).view(B, T, self.num_heads,
30                ↪ self.d_k).transpose(1, 2)
31
32            attn = self.scaled_dot_product_attention(Q, K, V)
33
34            attn = attn.transpose(1, 2).contiguous().view(B, T, -1)
35            return self.W_o(attn)

```

Listing 6: Minimal Implementation of Multi-Head Self-Attention

To obtain a fixed-size representation suitable for classification, token-level embeddings are aggregated using mask-aware mean pooling. Specifically, embeddings corresponding to padded positions are excluded from the average, ensuring that sequence length variation does not bias the representation. The pooled sentence representation is then passed through a linear classifier, which outputs logits corresponding to the sentiment classes.

The model is trained using the cross-entropy loss function, which is appropriate for multi-class classification tasks. Optimization is performed using the AdamW optimizer, which combines adaptive learning rates with decoupled weight decay to improve generalization. The training loop iterates over mini-batches of data, computes forward passes through the model, backpropagates the loss, and updates the model parameters. Performance is evaluated periodically on a validation set to monitor convergence and detect overfitting.

Several practical challenges arise when training Transformer models for sentiment analysis. Overfitting is a common issue, particularly when training on small or moderately sized datasets. This is mitigated through the use of dropout within the Transformer layers and weight decay in the optimizer. Another important consideration is computational cost. Self-attention has quadratic complexity with respect to sequence length, which can lead to increased memory usage and slower training for long inputs. As a result, careful selection of maximum sequence length and batch size is necessary to balance performance and efficiency.

The full implementation of this sentiment analysis pipeline, including the data pre-processing is provided in `code/sentiment_analysis.py`.

6 RNNs and LSTMs

Compare the implementation of Transformers with RNNs/LSTMs in PyTorch. Provide code snippets for both models and explain why Transformers outperform RNNs in handling long-range dependencies. Discuss computational trade-offs.

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were among the earliest neural architectures designed to handle sequential data. They process input tokens one step at a time, maintaining a hidden state that is updated recursively. In contrast, Transformers rely entirely on attention mechanisms and process sequences in parallel. In this section, we compare a Transformer-based sentiment classifier with an RNN-based classifier implemented in PyTorch, highlighting architectural differences, performance characteristics, and computational trade-offs.

The RNN-based sentiment classifier processes a sequence incrementally, updating its hidden state at each time step. Given an input sequence $(x_1, \dots, x_T)$, the hidden state h_t is computed as

$$h_t = f(h_{t-1}, x_t),$$

where f denotes the recurrent update function. In the case of LSTMs, this update includes gating mechanisms that regulate information flow through input, forget, and output gates, allowing the model to mitigate the vanishing gradient problem. After

```

1 class RNNClassifier(nn.Module):
2     def __init__(self, vocab_size, embed_dim=128, hidden_dim=128,
3         ↪ num_classes=2):
4         super().__init__()
5         self.embedding = nn.Embedding(vocab_size, embed_dim,
6             ↪ padding_idx=0)
7         self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
8         self.fc = nn.Linear(hidden_dim, num_classes)
9
10    def forward(self, x, mask):
11        # x: (batch, seq_len)
12        emb = self.embedding(x)
13        _, (h_n, _) = self.lstm(emb)
14        return self.fc(h_n[-1])

```

Listing 7: RNN/LSTM-Based Text Classifier

processing the full sequence, the final hidden state (or a pooled representation of all hidden states) is passed to a linear classifier to predict the sentiment label. The RNN classifier implementation is shown in Listing 7.

While LSTMs improve upon vanilla RNNs, they still suffer from inherent limitations when modeling long-range dependencies. Information must propagate sequentially through many time steps, making it difficult to preserve context over long sequences. This sequential dependency also prevents parallelization across time steps, resulting in slower training and inference.

The Transformer-based classifier, shown previously in Listing 4, addresses these limitations by using self-attention instead of recurrence. Rather than relying on a single hidden state that is updated over time, self-attention allows each token to directly attend to all other tokens in the sequence. As a result, long-range dependencies can be modeled more effectively, regardless of the distance between tokens. Additionally, the absence of recurrence enables full parallelization over sequence positions, significantly reducing training time on modern hardware.

From a computational perspective, the two architectures exhibit different trade-offs. RNNs and LSTMs have linear complexity with respect to sequence length, making them memory-efficient for long sequences. However, their lack of parallelism limits throughput and scalability. Transformers, on the other hand, incur a quadratic computational cost due to the self-attention mechanism, as every token attends to every other token. This increases memory usage and can become a bottleneck for very long sequences.

Despite this higher theoretical complexity, Transformers generally outperform RNNs and LSTMs in practice for sentiment classification and other NLP tasks. Their ability to model global context, combined with efficient parallel computation on GPUs, leads to faster convergence and better empirical performance. As a result, Transformer-based

models have largely replaced recurrent architectures as the dominant approach for handling long-range dependencies in modern sequence modeling tasks.

7 Transformer Optimizations

Discuss the computational challenges of implementing Transformers in PyTorch. Suggest optimization techniques such as gradient clipping, mixed precision training, and efficient batching. Provide code examples for at least two techniques.

While Transformer models offer strong performance across a wide range of NLP tasks, they also introduce significant computational challenges. The self-attention mechanism has quadratic complexity with respect to sequence length, leading to high memory usage and increased training time. In addition, deep Transformer architectures are prone to training instabilities such as exploding gradients. As a result, practical implementations in PyTorch often require optimization techniques to improve stability, efficiency, and scalability.

One common challenge when training Transformers is the presence of large gradient values, particularly in deep models or when using high learning rates. Exploding gradients can cause unstable updates and prevent convergence. Gradient clipping is a simple yet effective technique to address this issue by constraining the norm of the gradients during backpropagation. By enforcing an upper bound on the gradient norm, the model is prevented from making excessively large parameter updates, resulting in more stable training. The use of gradient clipping in the Transformer training loop is shown in Listing 8.

Another important optimization technique is mixed precision training. Transformers are computationally intensive and memory-hungry, making them well-suited for half-precision arithmetic. Mixed precision training combines 16-bit floating point operations with 32-bit precision where necessary, reducing memory consumption and improving throughput on modern GPUs. To avoid numerical instability, loss scaling is applied so that small gradient values are not lost due to reduced precision. PyTorch provides native support for mixed precision training through automatic casting and gradient scaling, as demonstrated in Listing 8.

In addition to these techniques, efficient batching plays a crucial role in optimizing Transformer performance. Grouping sequences of similar lengths within a batch reduces the amount of padding required, which in turn lowers unnecessary computation in the attention mechanism. This can lead to substantial speedups and better utilization of GPU memory, especially for large datasets with high variance in sequence length.

The relevant portions of code implementing gradient clipping and mixed precision training are provided in Listing 8. These optimizations are essential for training Transformer models reliably and efficiently, particularly when scaling to larger datasets and deeper architectures.

```

1 scaler = torch.cuda.amp.GradScaler()
2
3 for src, labels, mask in dataloader:
4     optimizer.zero_grad()
5
6     with torch.cuda.amp.autocast():
7         logits = model(src, mask)
8         loss = criterion(logits, labels)
9
10    scaler.scale(loss).backward()
11
12    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
13
14    scaler.step(optimizer)
15    scaler.update()

```

Listing 8: Optimizing Transformer Training with Gradient Clipping and Mixed Precision

8 Git Commit Log

```

92bc75e 2025-12-20 Nakul Bhat feat(report): add transformer optimization section
e69d46e 2025-12-20 Nakul Bhat feat(report): add rnn section
45fed85 2025-12-20 Nakul Bhat feat(report): add sentiment section
1c80c62 2025-12-20 Nakul Bhat feat(report): added multi self attention
6fe78d1 2025-12-20 Nakul Bhat feat(report): add sinpoe section
9915a65 2025-12-19 Nakul Bhat build(flake): add nltk
3c8df5e 2025-12-19 Nakul Bhat feat(report): add transformers section
04f7210 2025-12-19 Nakul Bhat feat(report): add seq2seq section
dedfb6b 2025-12-19 Nakul Bhat feat(report/listings): add seq2seq model
037c3a3 2025-12-19 Nakul Bhat refactor(code): cleanup
a22e5b3 2025-12-10 Nakul Bhat feat(report): add git history
4635a3f 2025-12-10 Nakul Bhat feat(report): add second page
814ffcb 2025-12-10 Nakul Bhat feat(report): setup packages
9452fdd 2025-12-10 Nakul Bhat feat(report): add titlepage
3b209ac 2025-12-09 Nakul Bhat feat(code): add sentiment analysis pipeline
1b7ce83 2025-12-09 Nakul Bhat build(flake): add tqdm
8be83c8 2025-12-08 Nakul Bhat feat(seq2seq): add seq2seq model
bb49af7 2025-12-08 Nakul Bhat build(flake): add matplotlib and pandas
8fab023 2025-12-05 Nakul Bhat feat(code): add seq2seq loader
0008289 2025-12-05 Nakul Bhat chore(gitignore): ignore data dir
d3338b9 2025-12-04 Nakul Bhat build(flake): update flake to add cuda
f66d0e4 2025-12-03 Nakul Bhat feat(report): start sections/seq2seq
9aaf0c7 2025-12-03 Nakul Bhat build(flake): add flake for pytorch
74915b0 2025-12-03 Nakul Bhat chore(gitignore): add python gitignore
4f871cf 2025-11-27 Nakul Bhat feat(readme): add note for visitors
8a6177f 2025-11-27 Nakul Bhat Initial commit

```

References

- [1] Kyle Aitken et al. *Understanding How Encoder-Decoder Architectures Attend*. Oct. 28, 2021. DOI: 10.48550/arXiv.2110.15253. arXiv: 2110.15253 [cs]. URL: <http://arxiv.org/abs/2110.15253>. Pre-published.
- [2] Dzmitry Bahdanau et al. “End-to-End Attention-Based Large Vocabulary Speech Recognition”. In: *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). Mar. 2016, pp. 4945–4949. DOI: 10.1109/ICASSP.2016.7472618. URL: <https://ieeexplore.ieee.org/abstract/document/7472618>.
- [3] *How to Use PyTorch for Sentiment Analysis on Textual Data? - GeeksforGeeks*. URL: <https://www.geeksforgeeks.org/deep-learning/how-to-use-pytorch-for-sentiment-analysis-on-textual-data/>.
- [4] Sean Robertson. *NLP From Scratch: Translation with a Sequence to Sequence Network and Attention — PyTorch Tutorials 2.9.0+cu128 Documentation*. URL: https://docs.pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html.
- [5] *Tatoeba: Collection of Sentences and Translations*. URL: <https://tatoeba.org/en>.
- [6] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.